

Claude Code on Desktop: Get Started with the Desktop App

Roteiro Completo de Curso Reformulado

Público-Alvo: Docentes e discentes iniciais da graduação em Engenharia

Objetivo Geral: Capacitar estudantes a usar Claude Desktop como ferramenta profissional de programação, com foco em segurança, integração com sistemas locais e exploração máxima do potencial da plataforma.

Pré-requisito: Conhecimento básico de programação (qualquer linguagem).

ESTRUTURA DO CURSO

Nível 1 (Iniciante): Módulos 0-16 — Fundamentos e uso profissional

Nível 2 (Avançado): Módulos 17-18 + Anexos — Automação e casos de uso avançado

Anexos: Troubleshooting, segurança, e referência técnica

MÓDULO 0: PRÉ-REQUISITOS E VERIFICAÇÃO INICIAL

Seção 0.1: Checklist Antes de Começar

Sugestão de Prints:

- Tela com informações de sistema (Windows: Configurações > Sistema > Sobre; macOS: Menu > Sobre este Mac)
- Print do site anthropic.com mostrando planos disponíveis

Descrição

Antes de instalar, você precisa confirmar que tem tudo que é necessário. Este módulo é um **pré-filtro** que economiza horas de frustrações.

Objetivos da Aula:

- Verificar compatibilidade do sistema operacional
- Confirmar qual plano você tem (Free vs Pro vs Max)
- Entender pré-requisitos técnicos (Git, espaço em disco, etc.)
- Saber exatamente para qual ajuda pedir se algo não funcionar

Habilidades Esperadas:

Ao final, você será capaz de:

- ☒ Identificar seu sistema operacional e arquitetura (x64, ARM64, Intel, Apple Silicon)
- ☒ Confirmar se seu plano Anthropic suporta Claude Desktop
- ☒ Verificar se tem Git instalado (Windows)
- ☒ Entender o que é esperado no seu caso específico

Seção 0.2: Verificação do Sistema Operacional

Sugestão de Prints:

- Windows: Configurações > Sistema > Sobre (destacar "Tipo de sistema")
- macOS: Menu > Sobre este Mac (destacar "Chip" ou "Processador")
- Linux: Resultado de `uname -a` no terminal

Conteúdo:

Sistemas Suportados Atualmente:

Sistema	Versão	Arquitetura	Suporte
Windows	10, 11+	x64 (Intel/AMD)	✔ Completo
Windows	10, 11+	ARM64 (Snapdragon, etc.)	✔ Completo
macOS	12+	Intel	✔ Completo
macOS	12+	Apple Silicon (M1, M2, M3+)	✔ Completo
Linux	Qualquer	Qualquer	✖ Não suportado (use CLI ou WSL2)

 Se você usa Linux:

Claude Desktop não funciona nativamente. Alternativas:

- Use Claude CLI (suportado em Linux)
 - Configure WSL2 (Windows Subsystem for Linux) se estiver em Windows
 - Use Docker com imagem oficial Claude
- Ver Módulo 19 para comparação Desktop vs CLI.

Como Verificar Seu Sistema:

Windows:

1. Clique em botão Start
2. Procure por "Configurações"
3. Vá para **Sistema > Sobre**
4. Procure por "Tipo de sistema" — será "x64" ou "ARM64"
5. Procure por "Versão do Windows" — deve ser 10 ou 11+

macOS:

1. Clique no menu Apple () no canto superior esquerdo
2. Escolha **Sobre este Mac**
3. Procure por "Chip" (Apple Silicon, ex: M1) ou "Processador" (Intel Core)
4. Procure por "Versão do macOS" — deve ser 12 ou superior

Seção 0.3: Verificação do Plano Anthropic

 Sugestão de Prints:

- Página de login Claude (claude.ai)
- Aba "Conta" mostrando "Seu plano: Pro" ou similar
- Mensagem "Upgrade Required" para alerta de plano incompatível

Conteúdo:

Quais Planos Permitem Claude Desktop?

Plano	Chat	Code	Cowork	Preço
Free	✓	✗	✗	Grátis
Pro	✓	✓	✗	~\$20/mês
Max	✓	✓	✓	~\$200/mês
Team	✓	✓	✓	Por usuário
Enterprise	✓	✓	✓	Customizado

⚠ Plano Free não suporta Desktop.

Se você vê mensagem "Upgrade Required" ao abrir a aba Code, seu plano não inclui acesso. Converse com seu professor sobre acesso institucional ou faça upgrade.

Como Verificar Seu Plano:

- 1. Vá para claude.ai
- 2. Faça login com sua conta Anthropic
- 3. Clique no ícone de **usuário** (canto superior direito)
- 4. Procure por **"Seu Plano"** ou **"Billing"**
- 5. Anotação: Se não ver opção Code, seu plano não suporta Desktop

Dica: Estudantes podem ter acesso gratuito ou com desconto via programa educacional. Pergunte ao seu professor ou consulte anthropic.com/education.

Seção 0.4: Pré-requisitos Técnicos

📷 Sugestão de Prints:

- Windows: Terminal PowerShell mostrando output de `git --version`
- macOS: Terminal mostrando output de `git --version`
- Explorador de arquivos mostrando espaço livre em disco

Conteúdo:

Requisitos Técnicos Mínimos:

- **Espaço em disco:** 500 MB livres (para aplicativo + cache)
- **RAM:** 4 GB mínimo (8 GB recomendado)
- **Internet:** Conexão estável (Claude Desktop funciona offline limitadamente)
- **Git (Windows apenas):** Necessário para sessões locais

Verificar Espaço em Disco:

- **Windows:** Clique em **Arquivo do Dispositivo** (Explorador) > selecione disco C: > veja "Espaço livre"
- **macOS:** Menu Apple > Sobre este Mac > **Armazenamento**

Verificar Se Git Está Instalado (Windows):

1. Abra **PowerShell** (procure no Start)
2. Digite: `git --version`
3. Se receber mensagem tipo "git version 2.40.0", já tem. ✅
4. Se não reconhecer comando, instale de git-scm.com (próxima seção)

Seção 0.5: Instalando Git (Windows Apenas)

Sugestão de Prints:

- Página de download git-scm.com com botão destacado "Download"
- Wizard de instalação Git mostrando opções padrão
- Confirmação final "Git foi instalado com sucesso"

Conteúdo:

Se você está em **Windows** e não tem Git, faça agora (levará 5 minutos):

1. Vá para git-scm.com
2. Clique em **Download for Windows**
3. Escolha **64-bit** (provavelmente seu sistema)
4. Execute o instalador
5. **Clique "Next" em todas as opções** (padrões estão ótimos)
6. Ao final, confirme "Git foi instalado com sucesso"

Verificar:

1. Abra novo PowerShell
2. Digite: `git --version`
3. Deve mostrar versão (ex: "git version 2.40.0") ✅

Pílula Hands-on: Seu Checklist Pessoal (5 minutos)

Preencha no papel ou editor:



```
Meu Sistema Operacional: [ ] Windows [ ] macOS [ ] Linux
Meu Tipo: [ ] x64 [ ] ARM64 [ ] Intel [ ] Apple Silicon
Meu Plano Anthropic: [ ] Free [ ] Pro [ ] Max [ ] Team
Tenho Git instalado (Windows)? [ ] Sim [ ] Não [ ] N/A
Tenho >500 MB livres? [ ] Sim [ ] Não
Tenho conexão estável? [ ] Sim [ ] Às vezes [ ] Não
```

Próximo passo: Se tudo está ✅, vá para Módulo 1. Se algo está ❌ e é bloqueante (tipo plano Free), resolva antes de continuar.

MÓDULO 1: O QUE É CLAUDE DESKTOP? (VISÃO GERAL)

Seção 1.1: Claude Desktop vs Web ([claude.ai](#))

Sugestão de Prints:

- Tela lado a lado: [claude.ai](#) web vs Claude Desktop app
- Interface do Desktop mostrando abas (Chat, Code, Cowork)
- Ícone do aplicativo na taskbar/dock

Descrição

O que mudou? Você conhece [claude.ai](#) (web). Claude Desktop é a **versão aplicativo** do mesmo Claude, mas com superpoderes para trabalhar com código.

Objetivos da Aula:

- Entender a diferença entre Web e Desktop
- Conhecer o que Desktop oferece que Web não oferece
- Entender por que Desktop é preferível para desenvolvimento

Habilidades Esperadas:

- ☒ Explicar as 3 diferenças principais entre Web e Desktop
- ☒ Saber quando usar Web e quando usar Desktop
- ☒ Identificar os 3 principais diferenciais do Desktop

Seção 1.2: Diferenças Principais

Conteúdo:

Aspecto	Web (claude.ai)	Desktop
Acesso a Arquivos Locais	❌ Não	☒ Sim
Velocidade	Normal	Mais rápido (menos latência)
Terminal Integrado	❌ Não	☒ Sim
Sessões Offline	❌ Não	⚠️ Limitado
Git Integration	❌ Não	☒ Sim

Integração com Arquivos	Cola manual	☒ Automática
Abas (Chat, Code, Cowork)	Parcial	☒ Todas

O Grande Diferencial: Acesso ao Sistema de Arquivos Local

Web ([claude.ai](#)):

Você: "Aqui está meu código" (copia manualmente)

Claude: (lê o que você colou)

Desktop:

Claude: (acessa todo seu projeto automaticamente)

Você: "Mude este arquivo"

Claude: (modifica direto no seu disco)

Isto muda **tudo**. Você não copia/cola. Claude vê seu projeto inteiro. Trabalha de verdade.

Seção 1.3: Casos de Uso Ideais para Desktop

Conteúdo:

Use Desktop quando:

- ☒ Trabalhando com projeto real (múltiplos arquivos)
- ☒ Precisa de terminal integrado
- ☒ Quer que Claude altere código automaticamente
- ☒ Trabalhando offline ou com latência alta

Use Web quando:

- ☒ Perguntando conceito rápido ("O que é REST API?")
- ☒ Sem acesso ao aplicativo
- ☒ Em computador compartilhado

Seção 1.4: As Três Abas: Chat, Code, Cowork

 Sugestão de Prints:

- Interface Desktop com as 3 abas visíveis no topo

- ABA Chat aberta
- ABA Code aberta (com projeto)
- ABA Cowork aberta

Conteúdo:

ABA Chat: Perguntas gerais, explicações conceituais

"Explique o que é um loop for"

"Qual é a diferença entre variável global e local?"

ABA Code: Trabalhar com arquivos reais do seu projeto

"Corrija os erros neste projeto"

"Adicione função para calcular média"

ABA Cowork: Agente automático que trabalha sozinho em background

"Execute testes, crie relatório, envie resumo — tudo sem supervisão"

Analogia: Chat é perguntar, Code é trabalhar na bancada, Cowork é contratar um assistente.

Pílula Hands-on: Identificar Casos de Uso (5 minutos)

Para cada tarefa abaixo, escreva qual aba você usaria (Chat, Code ou Cowork):

1. "Explique o que é um ponteiro em C"

Aba: _____

2. "Mude os 3 arquivos deste projeto para usar nomes em português"

Aba: _____

3. "Execute testes todas as noites e avise se falhar"

Aba: _____

4. "Me mostre um exemplo de construtor em Python"

Aba: _____

5. "Revise meu relatório de laboratório e sugira melhorias"

Aba: _____

Respostas esperadas: 1-Chat, 2-Code, 3-Cowork, 4-Chat, 5-Code

MÓDULO 2: INSTALAÇÃO SEGURA E VERIFICADA

Seção 2.1: Baixando o Instalador Correto

Sugestão de Prints:

- Página de download da Anthropic destacando botões "Download for Windows" e "Download for macOS"
- Dois instaladores lado a lado (Windows x64 vs ARM64)
- Página de checksum/verificação de integridade

Descrição

Escolher o instalador **correto** é crítico. Baixar a versão errada causa problemas de compatibilidade que parecem "o aplicativo não abre".

Objetivos da Aula:

- Baixar a versão correta para seu sistema
- Verificar integridade do download
- Preparar para instalação

Habilidades Esperadas:

- ☒ Identificar qual versão baixar baseado em seu sistema
- ☒ Verificar se download foi completo e seguro
- ☒ Localizar arquivo baixado

Seção 2.2: Qual Versão Escolher?

Sugestão de Prints:

- Página de download com todas as opções visíveis
- Destaque na escolha correta para seu sistema

Conteúdo:

Windows x64 (Intel ou AMD):



Se em Módulo 0 você viu "x64" → baixe este
Link: [anthropic.com/download](nome-do-arquivo-claude-desktop-x64.exe)
Tamanho: ~150-200 MB

Windows ARM64 (Snapdragon):



Se em Módulo 0 você viu "ARM64" → baixe este
Link: [anthropic.com/download](nome-do-arquivo-claude-desktop-arm64.exe)
Tamanho: ~150-200 MB

macOS Intel:



Se em Módulo 0 você viu "Intel" → baixe este
Link: [anthropic.com/download](nome-do-arquivo-claude-desktop-intel.dmg)
Tamanho: ~150-200 MB

macOS Apple Silicon (M1/M2/M3):



Se em Módulo 0 você viu "M1" ou "M2" ou "M3" → baixe este
Link: [anthropic.com/download](nome-do-arquivo-claude-desktop-apple-silicon.dmg)
Tamanho: ~150-200 MB

⚠ **Erro Comum:** Baixar "x64" em ARM64 = aplicativo não abre ou abre lentíssimo.

Solução: Se instalou errado, desinstale, baixe versão correta, reinstale.

Seção 2.3: Verificando Integridade do Download

Sugestão de Prints:

- Arquivo baixado na pasta Downloads
- Programa de hash aberto (7-Zip no Windows, Finder no macOS)
- Comparação visual entre hash esperado e calculado

Conteúdo:

Após baixar, **verifique se arquivo está completo e sem corrupção:**

Windows:

1. Abra PowerShell (procure no Start)
2. Navegue para pasta Downloads: `cd ~/Downloads`
3. Calcule hash: `certUtil -hashfile "nome-do-arquivo.exe" SHA256`
4. Compare com hash publicado no site da Anthropic
5. Se forem iguais , está seguro

macOS:

1. Abra Terminal (Aplicativos > Utilitários > Terminal)
2. Navegue para Downloads: `cd ~/Downloads`
3. Calcule hash: `shasum -a 256 nome-do-arquivo.dmg`
4. Compare com hash publicado
5. Se forem iguais , está seguro

Por que isso importa? Se arquivo foi corrompido durante download, instalação falhará de forma confusa. Verificar agora economiza horas de troubleshooting.

Seção 2.4: Instalação Passo a Passo

Sugestão de Prints:

Windows:

- Tela de bem-vindo do instalador
- Seleção de pasta de instalação (padrão: C:\Users[seu-usuario]\AppData\Local)
- Checkbox "Criar atalho no Desktop"
- Tela "Instalação Concluída"

macOS:

- Arquivo .dmg aberto (ícone Claude + pasta Applications)
- Arraste Claude para Applications (visual drag-drop)
- Tela de permissão de acesso

Conteúdo:

Windows (Instalador .exe):

1. Localize arquivo baixado na pasta **Downloads**
2. **Duplo clique** em arquivo `.exe`
3. Tela de segurança pode aparecer ("Você quer executar este arquivo?") → clique **Sim**
4. Tela de bem-vindo aparece → clique **Next**
5. Leia licença (se quiser) → clique **Aceitar**
6. Escolha pasta (padrão está ótimo) → clique **Next**
7. Marca "**Criar atalho no Desktop**" (facilita depois) → clique **Next**
8. Aguarde 2-3 minutos enquanto instala
9. Tela "Instalação Concluída" aparece → clique **Finalizar**
10. Atalho Claude aparece no seu Desktop 

macOS (Instalador .dmg):

1. Localize arquivo baixado na pasta **Downloads**
2. **Duplo clique** no arquivo `.dmg`
3. Janela abre mostrando ícone **Claude** e pasta **Applications**
4. **Arraste ícone Claude** para pasta Applications (copiã automático)
5. Pode parecer que nada acontece — aguarde 10-30 segundos

6. Quando terminar, ícone Claude aparece em Applications 
7. Eject o arquivo .dmg (opcional)

Seção 2.5: Primeiro Launch e Login

Sugestão de Prints:

- Tela de abertura Claude Desktop (splash screen)
- Tela de login/autenticação Anthropic
- Redirecionamento para navegador
- Tela de sucesso "Você está conectado"

Conteúdo:

Primeira vez que abre:

1. Duplo-clique no atalho **Claude** (Desktop)
2. Aplicativo abre (pode levar 10-30 segundos, primeira vez é mais lenta)
3. Tela pede para fazer login
4. Digite seu **email Anthropic** (mesmo de claude.ai)
5. Digite sua **senha**
6. Navegador abre para verificação adicional (segurança)
7. Confirme login no navegador
8. Volta para aplicativo, agora está conectado 
9. Você vê as três abas: **Chat**, **Code**, **Cowork**

Dica: Se login funciona no navegador mas não no Desktop, limpe cache:

Windows: Delete pasta `%APPDATA%\Claude`

macOS: Delete `~/ .cache/Claude`

Reinicie aplicativo.

Seção 2.6: Verificação Pós-Instalação

Sugestão de Prints:

- Menu do aplicativo mostrando Versão (Help > About)
- Abas (Chat, Code, Cowork) visíveis
- Mensagem de boas-vindas personalizada

Conteúdo:

Após login bem-sucedido, verifique:

Checklist de Verificação:

- ☐ **Aba Chat aberta:** **Aba Chat aberta:** Vejo caixa de texto, consigo escrever
- ☐ **Aba Code:** **Aba Code:** Vejo opção para selecionar projeto (pode estar vazia)
- ☐ **Aba Cowork:** **Aba Cowork:** Vejo interface para agentes (pode estar vazia)
- ☐ **Menu Help > About:** **Menu Help > About:** Vejo versão (ex: "1.0.0")
- ☐ **Sem mensagens de erro:** **Sem mensagens de erro:** Nenhuma notificação vermelha

Se todas estão , instalação foi bem-sucedida. Se alguma está , veja **Anexo A: Troubleshooting**.

Pílula Hands-on: Instalação e Primeiro Login (10 minutos)

1. **Baixe versão correta** (baseado em Módulo 0)
2. **Verifique hash** (se quiser ser paranóico)
3. **Instale** seguindo passos acima para seu SO
4. **Abra aplicativo**
5. **Faça login** com sua conta Anthropic
6. **Tire screenshot** da tela de boas-vindas

Envie para seu professor: Uma captura de tela mostrando Claude Desktop aberto e conectado.

MÓDULO 2B: SEGURANÇA DE DADOS LOCAIS

Seção 2B.1: O Que Claude Pode Ver

Sugestão de Prints:

- Estrutura de pasta do projeto (5-10 arquivos visíveis)
- Arquivo .env destacado (com aviso visual)
- Confirmação de permissões quando abre pasta
- Diff view mostrando acesso a conteúdo do arquivo

Descrição

Fato crítico: Quando você abre uma pasta em Claude Desktop, ele pode **ver e ler todos os arquivos** dentro dela.

Isto é poder, mas também responsabilidade.

Objetivos da Aula:

- Entender quais arquivos Claude acessa
- Aprender a proteger credenciais
- Configurar segurança mínima necessária

Habilidades Esperadas:

- ☒ Identificar arquivos sensíveis (credenciais, dados privados)
- ☒ Saber onde colocar segredos de forma segura
- ☒ Usar `.env` e `.gitignore` corretamente

Seção 2B.2: Arquivos Que Claude Pode Ver

Sugestão de Prints:

- Pasta de projeto aberta em explorador (Windows) ou Finder (macOS)
- Arquivo .env aberto no editor
- Arquivo .gitignore aberto mostrando padrões

Conteúdo:

Quando você seleciona uma pasta em Claude Desktop (aba Code > "Open Project"), Claude pode:

- ☒ **Ver (ler):**

- Todos os arquivos de texto: `.py`, `.js`, `.html`, `.md`, etc.
- Estrutura de pastas
- Nomes de arquivos e pastas

✗ NÃO pode:

- Modificar arquivo que você não pediu
- Acessar fora da pasta selecionada
- Ver sua senha ou arquivos do sistema

Seção 2B.3: Protegendo Credenciais

📷 Sugestão de Prints:

- Arquivo `.env` com exemplo de credenciais
- Arquivo `.env` listado em `.gitignore`
- Terminal mostrando `git status` (ignorando `.env`)

Conteúdo:

Cenário perigoso:



```
seu-projeto/  
├── app.py  
├── .env (⚠️ CONTÉM: DB_PASSWORD=senha123)  
└── requirements.txt
```

Você abre essa pasta em Claude, pede "corrija erros no banco de dados", Claude lê `.env` e vê a senha. Risco!

Solução: Use variáveis de ambiente

1. Crie arquivo `.env` (não compartilhado):



```
DB_HOST=localhost  
DB_PASSWORD=minha_senha_super_secreta  
API_KEY=abc123xyz
```

2. Adicione `.env` ao `.gitignore`:



```
# .gitignore
.env
.env.local
*.key
*.pem
```

3. No seu código, carregue variáveis:



```
# Python exemplo
import os
from dotenv import load_dotenv

load_dotenv()
db_password = os.getenv("DB_PASSWORD")
```

4. Quando pede ajuda a Claude:



Não envie `.env`. Claude pode ver, mas basta.
Se estiver seguro, ainda assim:
Peça: "Ajude com banco de dados, dados de conexão estão em `.env`"
Claude não toca `.env` sem você pedir explicitamente.

Seção 2B.4: Permissões e Escopo de Acesso

Sugestão de Prints:

- Tela de seleção de projeto (Open Project)
- Popup pedindo confirmação de acesso ("Claude quer acessar pasta X")
- Lista de arquivos que Claude pode ver (sidebar do Code)

Conteúdo:

Boa prática: Use subfolder seguro

Em vez de abrir todo seu usuário (`/Users/seu-nome`), abra subfolder específico:



```
Seu computador/  
├── Users/seu-nome/  
│   ├── [arquivos pessoais - NÃO abra]  
│   └── projetos/  
│       ├── calculadora-python/ ← ABRA ESTE  
│           ├── main.py  
│           ├── test.py  
│           └── .env
```

Por que? Claude vê apenas dentro de `calculadora-python/`. Arquivos pessoais, downloads, etc. estão protegidos.

Como fazer:

1. Aba Code > "Open Project"
2. Navegue para subfolder específico (ex: `~/projetos/meu-projeto`)
3. Clique "Select"
4. Claude agora vê apenas esse projeto

Seção 2B.5: Checklist de Segurança Antes de Compartilhar Pasta

Conteúdo:

Antes de abrir um projeto com Claude, faça este checklist:



```
[ ] Arquivo .env existe? Se sim:  
    [ ] Contém senhas/chaves API?  
    [ ] Está em .gitignore?  
    [ ] Posso deletar sem problema para teste?  
  
[ ] Tem arquivo .key ou .pem (chaves criptográficas)?  
    [ ] Está em .gitignore?  
  
[ ] Tem arquivo com dados pessoais (contatos, emails)?  
    [ ] Posso mover para fora da pasta?  
  
[ ] Este é subfolder específico?  
    [ ] Ou estou abrindo toda minha máquina?
```

Se respondeu "não" em todos os blocos de risco, está seguro.

Pílula Hands-on: Estrutura Segura (10 minutos)

1. **Crie pasta de teste:** /sua-home/projetos/test-seguro

2. **Crie arquivo** .env :

```
SECRET_KEY=nao_compartilhe_isto
DB_PASSWORD=senha123
```

3. **Crie arquivo** .gitignore :

```
.env
```

4. **Crie arquivo** .py simples:

```
import os
print("Projeto seguro")
```

5. **Tire screenshot** da estrutura de pasta

6. **Envie para professor:** Demonstrando que sabe proteger dados

MÓDULO 2C: INTEGRAÇÃO COM GIT E CONTROL DE VERSÃO

Seção 2C.1: Por Que Git Importa em Claude Desktop

Sugestão de Prints:

- Repositório Git inicializado (pasta .git)
- Status do Git mostrando mudanças (git status)
- Diff do Git (git diff)
- Histórico de commits (git log)

Descrição

Claude Desktop trabalha **melhor** quando seu projeto é um repositório Git. Aqui está por quê.

Objetivos da Aula:

- Entender relação entre Git e Claude Desktop
- Inicializar repositório Git
- Usar Git para rastrear mudanças de Claude

Habilidades Esperadas:

- ☒ Inicializar Git em um projeto
- ☒ Fazer commit de mudanças
- ☒ Entender o que Claude fez (via diff)
- ☒ Reverter mudanças se necessário

Seção 2C.2: Git Initialization

Sugestão de Prints:

- Terminal mostrando `git init`
- Pasta com .git criada
- `git status` mostrando arquivos untracked

Conteúdo:

Se seu projeto **NÃO** tem Git:



```
cd seu-projeto
git init
git add .
git commit -m "Initial commit"
```

Se seu projeto JÁ tem Git:

Bom! Você está preparado. Claude respeitará repositório existente.

Por que? Quando Claude faz mudanças, você consegue ver exatamente o que mudou:



```
git diff
```

Isto é segurança. Você revisa antes de aceitar.

Seção 2C.3: Rastreando Mudanças de Claude

Sugestão de Prints:

- Terminal mostrando `git status` após Claude fazer mudança
- Arquivo modificado marcado (M)
- `git diff` mostrando antes/depois
- `git log` com commits de Claude

Conteúdo:

Fluxo recomendado:

1. Abra projeto em Claude Desktop
2. Peça mudança: "Corrija erros no `main.py`"
3. Claude faz mudança
4. **No terminal:** `git diff` (revise mudanças)
5. Se aprovado: `git add .` e `git commit -m "Claude: fix errors in main.py"`
6. Se rejeitado: `git checkout .` (desfaça)

Exemplo visual:



```
$ git status
M main.py          ← modificado por Claude
A novo_arquivo.py ← novo arquivo criado por Claude

$ git diff main.py
- return sum / len(numeros) # ❌ errado
+ return sum(numeros) / len(numeros) # ✅ correto

# Aprova mudança:
$ git add .
$ git commit -m "Claude: fix calculation in main.py"
```

Pílula Hands-on: Git + Claude (5 minutos)

1. Crie pasta de teste:



```
mkdir projeto-test
cd projeto-test
```

2. Inicialize Git:



```
git init
git config user.name "Seu Nome"
git config user.email "seu@email.com"
```

3. Crie arquivo Python simples:



```
def saudar(nome):
    print(f"Olá, {nome}")
```

4. Commit inicial:



```
git add .
git commit -m "Initial commit: simple greeting function"
```

5. Verifique:



```
git log
```

Você deve ver seu commit.

MÓDULO 3: AS TRÊS ABAS (CHAT, CODE, COWORK)

Seção 3.1: Aba Chat - Perguntas e Explicações

Sugestão de Prints:

- Aba Chat aberta com conversa
- Botão para enviar mensagem
- Resposta de Claude mostrando explicação

Descrição

A aba **Chat** é como usar claude.ai, mas dentro do aplicativo Desktop.

Objetivos da Aula:

- Usar Chat para perguntas rápidas
- Entender limitações (sem acesso a arquivos)
- Saber quando Chat é suficiente vs quando precisa Code

Habilidades Esperadas:

- ☒ Fazer perguntas claras em Chat
- ☒ Receber explicações sem modificar código
- ☒ Saber quando mudar para aba Code

Seção 3.2: Quando Usar Chat

Conteúdo:

Chat é ideal para:

- Explicações conceituais: "O que é um algoritmo recursivo?"
- Comparações: "Qual é a diferença entre lista e tupla?"
- Brainstorming: "Como eu estruturaria um jogo de xadrez?"
- Dúvidas de sintaxe: "Como uso compreensão de lista em Python?"

Chat **NÃO** acessa:

- Seus arquivos locais
 - Seu projeto
 - Seu sistema
-

Seção 3.3: Aba Code - Trabalhando com Arquivos Reais

Sugestão de Prints:

- Aba Code aberta
- Projeto selecionado (pasta com vários arquivos)
- Lista de arquivos no sidebar
- Editor mostrando conteúdo de arquivo

Conteúdo

A aba **Code** é onde a magia acontece. Aqui Claude:

-  Vê todos os arquivos do seu projeto
-  Entende relacionamento entre arquivos
-  Pode modificar código (com sua permissão)
-  Trabalha com terminal integrado

Seção 3.4: Quando Usar Code

Conteúdo:

Code é ideal para:

- Corrigir erros em arquivo real: "Há um bug em [main.py](#), corrija"
- Refatoração: "Melhore a estrutura desta classe"
- Adicionar funcionalidade: "Crie função para validar email"
- Gerar código: "Crie arquivo de testes para esta função"

Code sempre:

- Acessa todo o projeto
- Mostra mudanças em diff view
- Pede permissão antes de aceitar mudanças (por padrão)

Seção 3.5: Aba Cowork - Agente Automático (Introdução)

Sugestão de Prints:

- Aba Cowork aberta
- Interface para criar tarefa
- Tarefa em progresso
- Resultado da tarefa concluída

Conteúdo

A aba **Cowork** é avançada. Claude trabalha **sozinho** em background, sem supervisão contínua.

Analogia: Chat é fazer pergunta. Code é trabalhar junto. Cowork é contratar alguém.

Exemplo:



```
Você: "Execute todos os testes, gere relatório HTML, envie email"
Claude (Cowork): Faz tudo. Você volta em 10 minutos, relatório pronto.
```

Para iniciantes: Skip Cowork por enquanto. Volte em Módulo 16+ quando estiver confortável.

Pílula Hands-on: Experiência com Três Abas (10 minutos)

1. Abra Chat:

- Pergunta: "O que é paradigma funcional?"
- Leia resposta
- Note que Claude não acessa seus arquivos

2. Vá para Code:

- Selecione um projeto pequeno (ou crie um novo)
- Crie arquivo `hello.py`:



```
def greet(name):
    return f"Hello, {name}"
```

- Peça: "Melhore esta função, adicione documentação"
- Revise mudanças em diff view
- Aceite ou rejeite

3. Veja Cowork:

- Clique na aba
- Note interface
- Por enquanto, não crie tarefa

MÓDULO 4: ESCOLHENDO SEU PROJETO

Seção 4.1: Conceito de "Projeto" em Claude Desktop

Sugestão de Prints:

- Diálogo "Open Project" mostrando navegação de pastas
- Diferentes estruturas de projeto (simples vs complexa)
- Confirmação de seleção

Descrição

Em Claude Desktop, você trabalha com **projetos** (pastas no seu computador).

Objetivos da Aula:

- Entender o que é um projeto
- Escolher primeiro projeto seguro
- Entender arquitetura mínima

Habilidades Esperadas:

- ☒ Selecionar pasta de projeto
- ☒ Entender estrutura básica
- ☒ Conhecer limitações de projeto

Seção 4.2: Qual Deve Ser Seu Primeiro Projeto?

Sugestão de Prints:

- Pasta simples com 3-5 arquivos
- Estrutura clara (sem pastas aninhadas muito profundas)

Conteúdo:

Primeira regra: Não comece com seu projeto mais importante.

Comece com algo:

- ☒ Pequeno (5-20 arquivos)
- ☒ Que você já entende
- ☒ Sem risco se Claude quebrar
- ☒ De preferência, sem dados sensíveis

Exemplos bons:

- Exercício de programação I
- Calculadora simples em Python
- Página HTML + CSS teste
- Script de utility (processamento de arquivo)
- Jogo simples (tipo snake, tic-tac-toe)

Evitar inicialmente:

- ❌ Projeto enorme (1000+ linhas)
- ❌ Código que você não entende
- ❌ Banco de dados com dados reais
- ❌ Projeto com dependências complicadas

Seção 4.3: Estrutura Mínima de Projeto

Sugestão de Prints:

- Estrutura simples de projeto (5 arquivos)
- Com [README.md](#)
- Com `.gitignore`
- Com arquivo principal

Conteúdo:

Projeto mínimo deve ter:

```
meu-projeto/
├── main.py           (ou main.js, main.cpp, etc.)
├── README.md         (explicação do que faz)
├── requirements.txt  (Python) ou package.json (JavaScript)
├── .gitignore        (o que não compartilhar)
└── test.py           (testes, se houver)
```

[README.md](#) deve ter:



```
# Meu Projeto

## O que faz
Calculadora simples que soma dois números.

## Como usar
python main.py

## Dependências
Nenhuma (ou liste aqui)
```

Claude lê README primeiro. Ajuda ele entender seu projeto.

Seção 4.4: Abrindo Projeto em Claude Desktop

Sugestão de Prints:

- Aba Code vazia
- Botão "Open Project" destacado
- Navegador de pastas aberto
- Projeto selecionado

Conteúdo:

Passo a passo:

1. Clique na **aba Code**
2. Clique em **"Open Project"** (ou ícone de pasta)
3. Navegue até seu projeto
4. Selecione a **pasta raiz** do projeto (não arquivo, pasta inteira)
5. Clique **"Select"** ou **"Abrir"**
6. Claude carrega projeto (lê todos os arquivos)
7. Após 2-5 segundos, projeto está pronto 

Você saberá que funcionou quando:

- Lista de arquivos aparece no sidebar esquerdo
- Você pode clicar em arquivos e ver conteúdo
- Pode escrever comando para Claude

Pílula Hands-on: Crie e Abra Projeto (15 minutos)

1. Crie pasta:

```
mkdir projeto-primeiro
cd projeto-primeiro
```

2. Crie arquivo **main.py**:

```
def somar(a, b):
    """Soma dois números"""
    return a + b

if __name__ == "__main__":
    print(somar(5, 3)) # Output: 8
```

3. Crie **README.md**:

```
# Meu Primeiro Projeto

Programa simples que soma dois números.

## Como usar
python main.py
```

4. Crie **.gitignore**:

```
__pycache__
.DS_Store
```

5. Crie Git repo:

```
git init
git add .
git commit -m "Initial commit"
```

6. Abra em Claude Desktop:

- Aba Code
- Open Project
- Selecione pasta **projeto-primeiro**
- Aguarde carregamento

7. Tire **screenshot** mostrando projeto aberto em Claude

MÓDULO 4B: INTEGRAÇÃO COM IDES (VS Code, PyCharm, Xcode)

Seção 4B.1: Por Que Integrar com IDE?

Sugestão de Prints:

- VS Code aberto com projeto
- Claude Desktop aberto lado a lado
- Mesmo projeto em ambos
- Fluxo de trabalho integrado

Descrição

Usar Claude Desktop em janela separada é ineficiente. IDEs modernas têm extensões que integram Claude **direto** no seu editor.

Objetivos da Aula:

- Entender ganho de integração
- Instalar extensão para sua IDE
- Usar Claude sem sair do editor

Habilidades Esperadas:

- ☒ Instalar extensão Claude na sua IDE
- ☒ Usar Claude a partir do editor
- ☒ Manter fluxo de trabalho fluído

Seção 4B.2: Claude Extension para VS Code

Sugestão de Prints:

- VS Code Marketplace buscando "Claude"
- Extensão instalada
- Ícone de Claude no sidebar
- Chat integrado no editor

Conteúdo:

VS Code é mais comum em Engenharia. Aqui está como:

1. **Abra VS Code**

2. Vá para **Extensions** (Ctrl+Shift+X / Cmd+Shift+X)
3. Procure por "**Claude**" ou "**Anthropic**"
4. Escolha a extensão oficial da Anthropic
5. Clique **Install**
6. Aguarde 1 minuto
7. Extensão aparece no sidebar esquerdo 

Depois:

- Clique no ícone Claude no sidebar
- Faça login com conta Anthropic
- Agora pode usar Claude direto no VS Code

Vantagem: Você vê seu código à esquerda, Claude à direita. Sem Alt+Tab.

Seção 4B.3: Claude Plugin para PyCharm

 Sugestão de Prints:

- PyCharm Settings > Plugins
- Buscando Claude
- Plugin instalado
- Claude Tool Window ativo

Conteúdo:

Para PyCharm (IDEs JetBrains):

1. **Abra PyCharm**
2. Vá para **File > Settings > Plugins** (ou **PyCharm > Preferences > Plugins** no macOS)
3. Clique **Marketplace**
4. Procure por "**Claude**" ou "**Anthropic Claude**"
5. Instale plugin oficial
6. Reinicie PyCharm
7. Nova aba "Claude" aparece na direita 

Depois:

- Clique na aba Claude
 - Faça login
 - Use como em VS Code
-

Seção 4B.4: Usando Claude Integrado

 Sugestão de Prints:

- Código no editor
- Seleção de texto
- Menu de contexto com opção "Ask Claude"
- Resposta aparecendo no painel

Conteúdo:

Fluxo de trabalho típico:

1. **Você escreve código no editor**
2. **Seleciona trecho problemático** (ou todo arquivo)
3. **Clica botão direito > Ask Claude** (ou atalho)
4. **Claude responde no painel** (sem sair do editor)
5. **Aceita sugestão ou continua editando**

Vantagem:

- Sem contexto perdido
- Sem janelas flutuantes
- Ambiente profissional
- Ganho de produtividade ~40%

Seção 4B.5: Alternativa: Claude Desktop + IDE lado a lado

Sugestão de Prints:

- Duas janelas: VS Code à esquerda, Claude Desktop à direita
- Mesma resolução (monitor panorâmico)

Conteúdo:

Se extensão não funcionar ou você preferir, use **lado a lado**:

1. **Redimensione janelas**: VS Code toma metade esquerda, Claude Desktop metade direita
2. **Alt+Tab rápido** se monitor pequeno
3. **Mesmo projeto em ambos**
4. Trabalhe normalmente

Não é tão fluído, mas funciona.

Pílula Hands-on: Integrar Claude na Sua IDE (10 minutos)

1. **Abra sua IDE** (VS Code, PyCharm, ou outra)
2. **Procure extensões/plugins Claude**
3. **Instale** (ou skip se não disponível)
4. **Faça login** com conta Anthropic

5. **Tire screenshot** mostrando Claude integrato

MÓDULO 4C: CONFIGURANDO .claude.md

Seção 4C.1: O Arquivo de Configuração

Sugestão de Prints:

- Arquivo .claude.md no projeto
- Conteúdo exemplo com regras
- Aba Code mostrando arquivo carregado

Descrição

O arquivo `.claude.md` é configuração que **você cria uma vez** e Claude **respeita sempre** quando trabalha no projeto.

Objetivos da Aula:

- Entender poder do `.claude.md`
- Criar arquivo para seu projeto
- Definir regras que Claude obedece

Habilidades Esperadas:

- ☒ Criar `.claude.md`
- ☒ Entender sintaxe básica
- ☒ Escrever regras úteis

Seção 4C.2: Quando Usar .claude.md

Conteúdo:

`.claude.md` é útil quando:

- ☒ Projeto tem **padrões específicos** (ex: "sempre usar TypeScript strict mode")
- ☒ Projeto usa **ferramentas específicas** (ex: "este projeto usa Poetry, não pip")
- ☒ Você quer **instruções permanentes** (ex: "código deve ter testes")
- ☒ Trabalha em **grupo** (todos têm mesmas regras)
- ☒ Projeto tem **restrições de segurança** (ex: "nunca usar eval()")

Seção 4C.3: Criando .claude.md

Sugestão de Prints:

- Arquivo `.claude.md` criado na raiz do projeto
- Conteúdo exemplo
- Claude lendo arquivo (confirmação no chat)

Conteúdo:

Crie arquivo `.claude.md` na raiz do seu projeto:



```
# Project Configuration for Claude

## Coding Standards

- Use Python 3.10+
- All functions must have docstrings
- Use type hints for function arguments and return values
- Follow PEP 8 naming conventions

## Testing Requirements

- All new functions must have tests
- Tests go in `test_*.py` files
- Run tests with: `pytest`

## Project Structure
```

projeto/ |—— src/ # Source code |—— tests/ # Test files |—— docs/ # Documentation |—— .claude.md # This file



Tools and Dependencies

- Language: Python 3.10
- Package Manager: Poetry
- Testing: pytest
- Linting: black + flake8

Important Rules

- Do NOT use eval() or exec()
- Do NOT hardcode passwords (use .env)
- Do NOT import from outside src/ and tests/
- Always add error handling

Common Commands

```
```bash
poetry install # Install dependencies
poetry run pytest # Run tests
poetry run black . # Format code
```

## Communication Style

---

- Explain complex decisions
- Suggest alternatives when appropriate
- Warn about potential issues
- Ask for clarification if ambiguous

## Notes

---

- This project follows [insert specific methodology]
- Database used: PostgreSQL
- API Framework: FastAPI
- Deployment: Docker containers



---

## ## Seção 4C.4: Como Claude Usa .claude.md

### ### Conteúdo:

Quando Claude abre seu projeto:

1. **Carrega** `.claude.md` automaticamente (se existir)
2. **Lê** todas as regras no arquivo
3. **Segue** as regras em toda interação
4. **Avisa** se regra seria quebrada (ex: "Código não seguiria padrão do projeto, ajuste")

**Exemplo:**

Você pede: "Crie função para validar email"

Claude lê `.claude.md`, vê que precisa:

- Docstring ☒ (rule: "functions must have docstrings")
- Type hints ☒ (rule: "use type hints")
- Teste ☒ (rule: "all functions must have tests")

Claude gera código seguindo **tudo**.

---

## ## Seção 4C.5: Evitando Erros Comuns

### ### Conteúdo:

**Não escreva:**



**Escreva:**

✗ `.claude.md` é super longo (>500 linhas) ✗ Regras contraditórias ✗ Instrução em linguagem natural vaga: "fazer tudo bom" ✗ Copiar regras de outro projeto sem adaptar

- ☒ Curto e objetivo (50-150 linhas)
- ☒ Regras claras e específicas
- ☒ Exemplos de código quando necessário
- ☒ Customizado para SEU projeto



---

## ## Pílula Hands-on: Crie .claude.md (10 minutos)

1. **Abra seu projeto** (em editor)
2. **Crie arquivo `.claude.md`** na raiz
3. **Copie template** acima (adapte para seu projeto)
4. **Salve**
5. **Feche projeto em Claude Desktop**
6. **Abra novamente** (força reload)
7. **Peça a Claude:** "Você carregou .claude.md?"
8. Claude deve responder afirmativamente

<!-- Quebra de Página -->

<div style="page-break-after: always;"></div>

## # MÓDULO 5: SELEÇÃO DE MODELO (HAIKU, SONNET, OPUS)

### ## Seção 5.1: Entendendo os Modelos

#### ### 📷 Sugestão de Prints:

- Menu de seleção de modelo mostrando: Haiku, Sonnet, Opus
- Tabela comparativa visível
- Seleção de modelo destacada

### ## Descrição

Claude oferece três modelos: **Haiku**, **Sonnet**, **Opus**. Escolher o certo para cada caso.

#### ### Objetivos da Aula:

- Entender diferenças entre modelos
- Saber qual usar quando
- Otimizar uso (custo vs qualidade)

#### ### Habilidades Esperadas:

- ☒ Conhecer força/fraqueza de cada modelo
- ☒ Tomar decisão rápida sobre qual usar
- ☒ Trocar modelo se necessário

---

### ## Seção 5.2: Matriz de Decisão (Qual Modelo Usar?)

#### ### 📷 Sugestão de Prints:

- Tabela comparativa de modelos
- Menu de seleção com ícones
- Tempo de resposta vs qualidade (gráfico)

#### ### Conteúdo:

Tarefa	Haiku	Sonnet	Opus
-----	-----	-----	-----
<b>**Explicar conceito rápido**</b>	<input checked="" type="checkbox"/>	<input checked="" type="checkbox"/> <input checked="" type="checkbox"/>	<input checked="" type="checkbox"/> <input checked="" type="checkbox"/> <input checked="" type="checkbox"/>

<b>**Debug simples**</b>	✓	✓	✓	✓	✓
<b>**Refatoração grande**</b>	✗	✓	✓	✓	✓
<b>**Análise arquitetura**</b>	✗	✓	✓	✓	✓
<b>**Gerar código novo**</b>	✓	✓	✓	✓	✓
<b>**Code review profundo**</b>	✗	✓	✓	✓	✓
<b>**Escrever testes**</b>	✓	✓	✓	✓	✓

---

## ## Seção 5.3: Haiku - O Rápido

### ### Conteúdo:

#### **\*\*Haiku é:\*\***

- ⚡ **\*\*Mais rápido\*\*** (~1-2 segundos de resposta)
- 💰 **\*\*Mais barato\*\*** (~1x custo)
- 🎯 **\*\*Bom para tarefas simples\*\***

#### **\*\*Use Haiku quando:\*\***

- ✓ Precisa resposta rápida
- ✓ Tarefa é simples/direta
- ✓ Economizar tokens importa
- ✓ Explicação conceitual curta

#### **\*\*Evite Haiku para:\*\***

- ✗ Refatoração de 500+ linhas
- ✗ Decisões arquiteturais complexas
- ✗ Code review profundo

#### **\*\*Exemplo:\*\***

Você: "Qual é a diferença entre list e tuple em Python?" Haiku: (responde em 2 segundos com explicação clara)





---

## ## Seção 5.4: Sonnet - O Equilibrado

### ### Conteúdo:

#### **\*\*Sonnet é:\*\***

- ⚡ **\*\*Rápido\*\*** (~2-4 segundos)
- 💰 **\*\*Preço médio\*\*** (~3x Haiku)
- 🎯 **\*\*Bom para maioria das tarefas\*\***

#### **\*\*Use Sonnet quando:\*\***

- ✅ Tarefa é média complexidade
- ✅ Precisa de balanceamento velocidade/qualidade
- ✅ Trabalhando com 100-300 linhas de código

**\*\*Sonnet é "Goldilocks":\*\*** não muito rápido, não muito lento. Bom para maioria dos casos

#### **\*\*Exemplo:\*\***

Você: "Refatore este código, melhore eficiência" Sonnet: (analisa, sugere mudanças claras em 4 segundos)



---

## ## Seção 5.5: Opus - O Potente

### ### Conteúdo:

#### **\*\*Opus é:\*\***

- 🕒 **\*\*Mais lento\*\*** (~5-10 segundos)
- 💰 **\*\*Mais caro\*\*** (~10x Haiku)
- 🧠 **\*\*Melhor qualidade, reasoning mais profundo\*\***

#### **\*\*Use Opus quando:\*\***

- ✅ Tarefa é muito complexa
- ✅ Precisa de análise profunda
- ✅ Refatoração grande (500+ linhas)
- ✅ Decisão arquitetural importante

#### **\*\*Exemplo:\*\***

Você: "Analyze esta arquitetura de microserviços, identifique problemas" Opus: (análise profunda de 8 segundos, identifica 5 problemas subtis)



---

## ## Seção 5.6: Mudando de Modelo Rapidamente

### ### 📷 Sugestão de Prints:

- Menu de seleção de modelo próximo ao botão enviar
- Dropdown mostrando Haiku, Sonnet, Opus
- Mudança de modelo destacada

### ### Conteúdo:

**Como trocar de modelo:**

1. Veja o **menu de seleção** (próximo ao botão de envio na aba Code)
2. Clique e escolha: **Haiku**, **Sonnet**, **Opus**
3. A mudança toma efeito **imediatamente**
4. Próxima mensagem usa modelo novo

---

## ## Seção 5.7: Economizando Tokens

### ### Conteúdo:

**Dica profissional:**

Comece com **Haiku**. Se resposta não é boa, **upgrade para Sonnet**. Se Sonnet não res

Fluxo econômico:

1. Tente Haiku (rápido, barato)
2. Se não bom → Sonnet (melhor qualidade)
3. Se ainda não bom → Opus (melhor reasoning)



Economiza ~70% de tokens mantendo qualidade.

---

## ## Pílula Hands-on: Teste os Três Modelos (15 minutos)

1. **\*\*Aba Code, selecione seu projeto\*\***
2. **\*\*Mude para Haiku\*\***, faça pergunta simples: "Qual é a complexidade de merge sort?"
3. **\*\*Mude para Sonnet\*\***, mesma pergunta: "Qual é a complexidade de merge sort?"
4. **\*\*Mude para Opus\*\***, mesma pergunta: "Qual é a complexidade de merge sort?"
5. **\*\*Compare respostas\*\*** note velocidade e profundidade
6. **\*\*Tire screenshots\*\*** de cada resposta

<!-- Quebra de Página -->

<div style="page-break-after: always;"></div>

## # MÓDULO 6: ENGENHARIA DE PROMPTS

### ## Seção 6.1: O Que é um Prompt Bom?

#### ### 📷 Sugestão de Prints:

- Lado esquerdo: prompt ruim (genérico)
- Lado direito: prompt bom (específico)
- Comparação visual

### ## Descrição

**\*\*Prompt\*\*** é o que você escreve para Claude. Prompt bom = resposta boa. Prompt ruim = r

#### ### Objetivos da Aula:

- Escrever prompts efetivos
- Entender estrutura de prompt
- Corrigir prompts que não funcionam

#### ### Habilidades Esperadas:

- ☒ Transformar prompt vago em específico
- ☒ Incluir contexto suficiente
- ☒ Definir resultado esperado claramente
- ☒ Saber quando pedir refinamento

---

### ## Seção 6.2: Estrutura de Prompt Efetivo

#### ### Conteúdo:

**\*\*Prompt inefetivo\*\***



Claude não sabe o **quê** mudar, para **quê**, como.

***\*\*Prompt efetivo:\*\****

✓ Contexto: Este é um programa que calcula média de notas ✓ Tarefa: Encontre erros lógicos ✓ Limite: Não altere a interface de entrada/saída ✓ Formato: Mostre os erros em lista, com explicação de por que estão errados



---

***## Seção 6.3: Fórmula de Prompt Efetivo***

***### Conteúdo:***

***Use esta fórmula como template:***

CONTEXTO: [O que é este projeto? O que faz?]

TAREFA: [Exatamente o que você quer que Claude faça]

RESTRIÇÃO: [O que NÃO deve fazer ou alterar]

FORMATO: [Como você quer a resposta? (lista, código, explicação, etc.)]

NIVEL: [Explicar para iniciante ou assumir conhecimento avançado?]



***\*\*Exemplo **real**:***

CONTEXTO: Este é um programa em Python que lê arquivo CSV com vendas de 2024 e calcula estatísticas. Tem 150 linhas.

TAREFA: Encontre bugs na função `calcular_media()`. Esta função deveria retornar a média de valores na coluna "preço".

RESTRIÇÃO: Não mude nomes de variáveis. Não altere estrutura geral.

FORMATO: Explique cada bug encontrado. Mostre antes/depois. Use tipo iniciante.

NIVEL: Explique como se eu tivesse completado Python 101 (semestre 1).



---

## ## Seção 6.4: Melhorando Prompts Iterativamente

### ### Conteúdo:

**\*\*Se resposta não é boa:\*\***

1. **\*\*Leia o que Claude respondeu\*\***
2. **\*\*Identifique o problema:\*\*** muito genérico? muito técnico? perdeu contexto?
3. **\*\*Refine prompt:\*\*** adicione detalhe, contexto, restrição
4. **\*\*Envie novamente\*\***

**\*\*Exemplo:\*\***

❌ Tentativa 1: "Explique banco de dados" → Resposta: genérica demais

✅ Tentativa 2: "Explique como funciona índice em banco de dados SQL. Explique para alguém que sabe SQL básico mas não sabe otimização. Dê um exemplo com tabela de 1 milhão de registros." → Resposta: exatamente o que preciso



---

## ## Seção 6.5: Incluindo Contexto (Arquivos, Imagens, Descrições)

### ### 📷 Sugestão de Prints:

- Botão de anexar arquivo
- Imagem anexada ao prompt
- Referência a arquivo: "@nome\_do\_arquivo.py"

### ### Conteúdo:

**\*\*Se seu prompt precisa contexto:\*\***

- **\*\*Descreva arquivo:\*\*** "No arquivo `main.py`, linha 42..."
- **\*\*Anexe arquivo:\*\*** Clique botão paperclip, selecione arquivo
- **\*\*Use `@file.py`:\*\*** Escreva "@main.py" para mencionar arquivo específico
- **\*\*Anexe imagem:\*\*** Diagrama, screenshot, estrutura visual
- **\*\*Copie/cole trecho:\*\*** Último recurso (prefira anexar arquivo)

---

## ## Pílula Hands-on: Reescreva Prompts (10 minutos)

Transforme estes prompts ruins em bons:

1. **\*\*Ruim:\*\*** "Faça meu trabalho"  
**\*\*Bom:\*\*** (escreva um prompt efetivo)
2. **\*\*Ruim:\*\*** "Corrija erros"  
**\*\*Bom:\*\*** (escreva um prompt efetivo)
3. **\*\*Ruim:\*\*** "Melhore isto"  
**\*\*Bom:\*\*** (escreva um prompt efetivo)

Use a fórmula: Contexto → Tarefa → Restrição → Formato → Nível

<!-- Quebra de Página -->

<div style="page-break-after: always;"></div>

## # MÓDULO 7: REVISÃO ANTES DE ACEITAR (DIFF VIEW)

### ## Seção 7.1: Por Que Revisar Sempre

#### ### 📷 Sugestão de Prints:

- Diff view lado a lado: antes e depois
- Linhas adicionadas em verde
- Linhas removidas em vermelho

### ## Descrição

**\*\*Regra de ouro:\*\*** Nunca aceite mudança de Claude sem revisar. Mesmo que pareça certa.

Claude erra. Você tem responsabilidade de verificar.



### ### Objetivos da Aula:

- Entender diff view
- Revisar mudanças criticamente
- Aceitar ou rejeitar com confiança

### ### Habilidades Esperadas:

-  Ler diff view facilmente
-  Identificar mudanças perigosas
-  Entender impacto de cada mudança
-  Rejeitar mudanças inadequadas

---

## ## Seção 7.2: Entendendo Diff View

### ### 📷 Sugestão de Prints:

- Diff view mostrando: antes à esquerda, depois à direita
- Linhas adicionadas (verde)
- Linhas removidas (vermelho)
- Linhas modificadas (amarelo)

### ### Conteúdo:

#### **\*\*Exemplo:\*\***

```
```diff
def calcular_media(notas):
-     return sum / len(notas)           ← Removido (ERRADO)
+     return sum(notas) / len(notas) ← Adicionado (CORRETO)
```

Símbolos:

-  **Verde (+):** Linha adicionada
-  **Vermelho (-):** Linha removida
-  **Amarelo:** Linha modificada
-  **Cinza:** Linha não alterada (contexto)

Seção 7.3: Checklist de Revisão

Conteúdo:

Antes de aceitar mudança, pergunte-se:



```
[ ] Entendo o que mudou? (Posso explicar?)  
[ ] Mudança faz sentido? (Logicamente correta?)  
[ ] Quebra algo? (Outro arquivo dependeria de código antigo?)  
[ ] Segue padrões do projeto? (Type hints, nomes, estrutura?)  
[ ] Tem testes? (Se mudança grande, tem testes novos?)  
[ ] Está seguro? (Sem vulnerabilidades óbvias?)
```

Se responder "não" em qualquer:

1. Rejeite mudança (botão Reject)
2. Explique problema a Claude
3. Claude refaz

Seção 7.4: Rejeitando Mudanças



Sugestão de Prints:

- Botão "Reject" destacado
- Diálogo pedindo motivo da rejeição
- Mensagem clara de rejeição enviada a Claude

Conteúdo:

Se mudança está errada:

1. Clique **Reject** (ou **Don't Accept**)
2. Explique por quê: "Esta mudança quebra a função anterior porque..."
3. **Claude relê seu feedback e tenta novamente**

Exemplo:



Você: *"Rejeito esta mudança.*
Você removeu validação de email.
Agora qualquer *string* é aceita.
Reescreva mantendo validação."

Claude: (lê feedback, entende problema, refaz com validação)

Seção 7.5: Aceitar Mudanças

Sugestão de Prints:

- Botão "Accept" destacado
- Confirmação visual de aceitação
- Arquivo atualizado no projeto

Conteúdo:

Quando mudança está certa:

1. Clique **Accept** (ou **Apply**)
2. Arquivo é modificado no seu disco **automaticamente**
3. Se usando Git: faça commit

```
git add .  
git commit -m "Claude: fix calculation in media function"
```



Pílula Hands-on: Revisar uma Mudança (10 minutos)

1. **Aba Code, abra seu projeto**
2. **Peça a Claude:** "Adicione função para validar se email é válido"
3. **Claude sugere código**
4. **Examine diff view:**
 - Que linhas foram adicionadas?
 - Validação está correta?
 - Segue padrões do projeto?
5. **Aceite ou rejeite** com justificativa
6. **Tire screenshot** do diff view

MÓDULO 8: INTERRUPÇÃO E REDIRECIONAMENTO

Seção 8.1: Você Controla Claude

Sugestão de Prints:

- Botão "Stop" destacado durante execução
- Mensagem "Claude foi interrompido"
- Novo prompt para redirecionar

Descrição

Claude às vezes **começa errado**. Você não precisa esperar terminar. Pode interromper e redirecionar.

Objetivos da Aula:

- Saber quando interromper
- Usar interrupção estrategicamente
- Redirecionar sem perder progresso

Habilidades Esperadas:

- ☒ Identificar quando Claude está errado
- ☒ Interromper apropriadamente
- ☒ Dar feedback claro

Seção 8.2: Quando Interromper

Conteúdo:

Interrompa quando:

- Claude está gerando código muito complexo (você pediu simples)
- Claude deletaria código que você quer manter
- Claude está refatorando errado
- Claude esqueceu restrição que você deu

Não interrompa:

- Claude está apenas pensando (normal, leva 5 segundos)
 - Claude está formatando (normal, leva 10 segundos)
 - Você apenas está impaciente (deixe terminar)
-

Seção 8.3: Como Interromper

Sugestão de Prints:

- Mensagem em progresso de Claude
- Botão "Stop" ou "Cancel" brilhando
- Confirmação de interrupção

Conteúdo:

Durante resposta de Claude:

1. Clique botão **"Stop"** (ou **"Cancel"**)
2. Claude para imediatamente
3. Digite novo prompt: "Não, espera. Antes..."
4. Claude lê novo prompt e repensa

Seção 8.4: Redirecionando

Conteúdo:

Exemplo de redirecionamento:



Claude está criando função `"delete_all_users()"`.
Você: *"STOP! Não quero function delete."*
Ao invés, crie `function 'soft_delete(user_id)'` que apenas marca como inativo."

Claude: (relê, entende, cria `soft_delete` ao invés)

Pílula Hands-on: Praticar Interrupção (5 minutos)

1. **Abra projeto simples**
2. **Peça:** "Refatore este arquivo inteiro"
3. **Aguarde 5 segundos** enquanto Claude pensa
4. **Clique Stop**
5. **Redirecione:** "Não inteiro. Apenas a função `calcular_media()`"
6. **Claude refoca no que você pediu**

MÓDULO 9: FORNECIMENTO DE CONTEXTO

Seção 9.1: Mais Contexto = Melhor Resposta

Sugestão de Prints:

- Arquivo anexado ao prompt
- Referência @arquivo.py no texto
- Imagem anexada ao prompt
- Estrutura de pasta como contexto

Descrição

Claude trabalha **muito melhor** quando você fornece contexto suficiente.

Objetivos da Aula:

- Entender tipos de contexto
- Fornecer contexto efetivamente
- Usar anexos e referências

Habilidades Esperadas:

- ☒ Identificar contexto necessário
- ☒ Anexar arquivos corretamente
- ☒ Descrever contexto em texto
- ☒ Referenciar partes específicas

Seção 9.2: Tipos de Contexto

Conteúdo:

Contexto de arquivo:

"No arquivo @main.py, a função calcular() na linha 42..."



Contexto de imagem:



Diagrama de arquitetura (screenshot)
Sketch de interface (foto)
Erro de compilador (print)

Contexto verbal:



"Este projeto processa imagens. Recebe arquivo .jpg, saída .png"

Contexto de estrutura:



```
pasta/  
├── src/  
│   ├── main.py  
│   └── utils.py  
├── tests/  
│   └── test_main.py  
└── config.json
```

Seção 9.3: Dando Contexto Efetivamente

Conteúdo:

Contexto ruim:



✗ "Tem um erro no meu projeto"

Claude não sabe qual erro, em qual arquivo.

Contexto bom:



✓ "Este projeto processa CSVs de vendas.
Arquivo data/sales.csv tem 1 milhão de linhas.
Programa: lê CSV, calcula total por produto.
Erro: programa consome 8GB RAM, deveria ser <500MB.
Arquivo principal: src/main.py (anexado)
Versão Python: 3.10"

Seção 9.4: Anexando Arquivos

Sugestão de Prints:

- Botão de anexar (paperclip)
- Múltiplos arquivos anexados
- Visualização de arquivo anexado

Conteúdo:

Como anexar:

1. Clique botão  (paperclip) ou +
2. Selecione arquivo(s)
3. Arquivo aparece no prompt
4. Você pode digitar abaixo

Ou mencione arquivo:



```
"No arquivo @main.py, a linha 42..."
```

Claude carrega arquivo automaticamente.

Seção 9.5: Quando NÃO Anexar

Conteúdo:

NÃO anexe:

-  Arquivo `.env` com senhas
-  Arquivo grande (> 10MB)
-  Arquivo binário (imagem, audio)
-  Arquivo que Claude não precisa

Descreva ao invés:



```
"Arquivo de configuração define:  
DB_HOST=localhost  
DB_PORT=5432  
TABLE_USERS=usuarios"
```


Pílula Hands-on: Fornecer Contexto (10 minutos)

1. Crie dois prompts:

Prompt 1 (sem contexto):

"Tem um erro no meu código Python"



Prompt 2 (com contexto):

"Arquivo @main.py tem função 'validar_email()'.
Função deveria retornar True se email válido.
Atualmente retorna False para emails válidos.
Exemplo: 'user@example.com' → deveria ser True, mas retorna False"



2. Envie ambos a Claude
3. Compare respostas
4. Note diferença de qualidade

MÓDULO 10: MODOS DE PERMISSÃO (ASK, AUTO-ACCEPT, PLAN)

Seção 10.1: Controle de Mudanças

Sugestão de Prints:

- Menu de permissões destacado
- Três modos visíveis: Ask, Auto-accept, Plan
- Seleção de modo

Descrição

Você controla **como** Claude faz mudanças. Três modos disponíveis.

Objetivos da Aula:

- Entender três modos
- Escolher modo apropriado
- Trocar modos conforme necessário

Habilidades Esperadas:

- ☒ Explicar diferença entre modos
- ☒ Usar "Ask" para iniciantes
- ☒ Usar "Auto-accept" para tarefas repetitivas
- ☒ Usar "Plan" para decisões complexas

Seção 10.2: Ask Permissions (Padrão)

Conteúdo:

"**Ask Permissions**" é modo padrão. Claude:

1. Propõe mudança
2. **Você revisa em diff view**
3. Você clica Accept ou Reject

Melhor para:

- ☒ Iniciantes (seguro)
- ☒ Código importante (precisa revisar)
- ☒ Aprender (entende cada mudança)

- ☒ Primeiro contato com projeto

Analogia: Claude escreve relatório, você aprova antes de publicar.

Seção 10.3: Auto-Accept Edits

Conteúdo:

"**Auto-accept Edits**" aceita mudanças automaticamente. Claude:

1. Faz mudança
2. **Arquivo atualizado sem permissão**
3. Você só vê resultado

Melhor para:

- ☒ Tarefas repetitivas
- ☒ Você confia em Claude completamente
- ☒ Projeto com controle de versão (pode desfazer)
- ☒ Refatoração simples

⚠ **Cuidado:** Se Claude erra, arquivo já mudou. Você precisa desfazer (git revert).

Seção 10.4: Plan Mode

Conteúdo:

"**Plan Mode**" faz Claude planejar antes. Claude:

1. **Propõe plano** ("vou fazer A, depois B, depois C")
2. **Você aprova plano**
3. Claude executa
4. Você revisa resultado

Melhor para:

- ☒ Refatoração grande (500+ linhas)
- ☒ Mudança de arquitetura
- ☒ Decisão crítica
- ☒ Quando quer entender estratégia antes

Analogia: Arquiteto apresenta blueprints. Você aprova design. Depois constrói.

Seção 10.5: Mudando de Modo

🖼️ Sugestão de Prints:

- Menu de modos no topo ou sidebar
- Indicador visual de modo atual

Conteúdo:

Como trocar:

1. Procure menu de "**Permissions**" ou "**Mode**" (próximo a botão enviar)
2. Escolha: **Ask, Auto-accept, Plan**
3. Mudança toma efeito **imediatamente**

Pílula Hands-on: Testar Modos (10 minutos)

1. **Abra seu projeto**
2. **Defina modo "Ask Permissions"**
3. **Peça:** "Adicione docstring a todas as funções"
4. **Revise diff view, aceite**
5. **Mude para "Plan Mode"**
6. **Peça:** "Refatore main() para ser mais legível"
7. **Claude propõe plano. Você aprova ou rejeita.**
8. **Tire screenshots** de ambos os modos

MÓDULO 11: DIFF VIEW (APROFUNDADO)

Seção 11.1: Lendo Diff View Expertly

Sugestão de Prints:

- Diff view lado a lado (antes e depois)
- Linha por linha destacada
- Número de linhas em amarelo (+5, -2)

Descrição

Diff view é crítica. Você precisa **ler fluentemente**.

Objetivos da Aula:

- Ler diff view com confiança
- Identificar padrões perigosos
- Entender impacto de mudanças

Habilidades Esperadas:

- ☒ Ler diff view rapidamente
- ☒ Identificar tipos de mudança
- ☒ Avisar Claude de erros
- ☒ Compreender dependências

Seção 11.2: Tipos de Mudança a Observar

Conteúdo:

Mudanças boas:

```
- return sum / len(notas)          # ERRADO
+ return sum(notas) / len(notas)   # CORRETO
```



Mudanças que precisam verificação:



```
- def calcular(a, b):  
+ def calcular_soma(a, b): # Nome mudou!
```

Nome mudou em um lugar. Outras funções que chamam `calcular()` quebram? Verificar!

● Mudanças perigosas:



```
- if not user.verified:  
-     return None  
+ if not user.verified:  
+     return user # PERIGO! Deveria retornar None se não verificado
```

Seção 11.3: Checklist de Sinais de Alerta

Conteúdo:



🚨 SINAIS DE ALERTA em diff view:

- [] Função removida inteira (é usada em outro lugar?)
- [] Variável renomeada (outras referências quebram?)
- [] Lógica invertida (`if x` → `if not x`)
- [] Return type mudou (esperava `string`, agora `int`?)
- [] Loop removido (processamento ia ficar incompleto?)
- [] Exception handling removido (erros agora não tratados?)
- [] Magic number adicionado (`17`, `42`, etc. sem explicação?)

Se vir qualquer uma, **rejeite e peça explicação**.

Seção 11.4: Exemplo Completo de Revisão

Conteúdo:

Cenário: Você pediu: "Otimize função de busca"

Claude retorna diff:



```
def busca(lista, alvo):  
-     for i in range(len(lista)):  
-         if lista[i] == alvo:  
-             return i  
-     return -1  
+     try:  
+         return lista.index(alvo)  
+     except ValueError:  
+         return -1
```

Sua análise:

1. ☒ Faz a mesma coisa? Sim (retorna índice ou -1)
2. ☒ É mais rápido? Sim (usa built-in otimizado)
3. ☒ Segue padrões? Sim (Python idiomático)
4. ☒ Quebra algo? Não
5. ☒ Compreensível? Sim (melhor até)

Decisão: ACEITE ☒

Pílula Hands-on: Revisar Diff Complexa (10 minutos)

1. **Peça a Claude:** "Refatore a estrutura deste arquivo (10+ mudanças)"
2. **Claude mostra diff view grande**
3. **Você lê linha por linha:**
 - Mudanças fazem sentido?
 - Algo perigoso?
 - Arquivo inteiro ainda funciona?
4. **Tome decisão:** Aceitar ou rejeitar
5. **Se rejeitar, explique por quê**

MÓDULO 12: TERMINAL INTEGRADO

Seção 12.1: Terminal como Ferramenta

Sugestão de Prints:

- Terminal integrado aberto (Ctrl+`)
- Comando sendo executado
- Output do comando visível

Descrição

Claude Desktop tem **terminal integrado**. Você pode rodar comandos sem sair do aplicativo.

Objetivos da Aula:

- Entender terminal integrado
- Rodar comandos básicos seguramente
- Usar terminal com Claude

Habilidades Esperadas:

-  Abrir terminal
-  Executar comandos seguros
-  Entender quando pedir a Claude

Seção 12.2: Abrindo Terminal

Sugestão de Prints:

- Atalho de teclado (Ctrl + `)
- Terminal aparecendo na base do editor
- Prompt  pronto

Conteúdo:

Abrir terminal:

- **Windows:** Ctrl + ` (backtick)
- **macOS:** Cmd + ` (backtick)
- **Linux:** Ctrl + ` (dependendo de seu ambiente)

Ou procure no menu do aplicativo > "Terminal" ou "New Terminal".

Seção 12.3: Comandos Seguros vs Perigosos

Sugestão de Prints:

- Comando seguro: `python script.py`
- Comando perigoso: `rm -rf /`
- Confirmação de risco

Conteúdo:

Escala de Risco:

Seguro (execute livremente):

```
python script.py      # Rodar script
npm test              # Rodar testes
git status            # Ver estado do Git
ls                    # Listar arquivos
cat file.txt          # Ver conteúdo
```



Cuidado (você entende risco):

```
pip install library   # Instalar dependência
npm install            # Instalar dependência
git commit            # Commitar mudanças
```



Perigoso (nunca, sem razão forte):

```
rm file               # Deletar arquivo
rm -rf folder         # Deletar pasta inteira
mv file /outro/lugar  # Mover arquivo
```



NUNCA execute:

```
sudo rm -rf /          # Deletar TUDO
eval "code"            # Executar código desconhecido
```



Seção 12.4: Usar Terminal com Claude

Conteúdo:

Fluxo típico:

1. **Você pede a Claude:** "Execute testes"
2. **Claude sugere comando:** `pytest`
3. **Você copia e cola no terminal** (ou Claude executa se autorizado)
4. **Terminal mostra resultado**
5. **Se erro, mostra no terminal. Você mostra erro a Claude.**

Exemplo:



```
Você: "Execute testes, diga se passaram"

Claude: "Vou executar pytest. Cole isto no terminal:"
        pytest

Você: (cola no terminal)
        PASSED 15 / FAILED 2

Você para Claude: "Dois testes falharam. Aqui está erro:
                  AssertionError: expected 5, got 4"

Claude: (lê erro, sugere correção)
```

Seção 12.5: Pedir a Claude Fazer Algo no Terminal

Conteúdo:

Às vezes Claude executa comando **direto** no terminal integrado (se autorizado):



```
Você: "Teste se meu código está pronto para entregar"
Claude: "Vou executar:
        1. pytest (verificar testes)
        2. black --check . (verificar formatação)
        3. flake8 (verificar linting)"

[Claude executa automaticamente]

Claude: "Resultado:
        - Testes: ✅ 20 PASSED
        - Formatação: ✅ OK
        - Linting: ❌ 3 problemas em main.py linha 42"
```

Pílula Hands-on: Terminal Seguro (10 minutos)

1. **Abra terminal** (Ctrl + `)
2. **Execute comando seguro:** `ls` ou `dir` (lista arquivos)
3. **Navegue:** `cd seu-projeto`
4. **Veja Git status:** `git status`
5. **Rode testes (se existirem):** `pytest` ou similar
6. **Tire screenshot** do resultado

MÓDULO 13: PREVIEW

Seção 13.1: Vendo Aplicação Rodando

Sugestão de Prints:

- Aplicação em preview (navegador embarcado)
- Página web funcionando
- Claude ao lado analisando

Descrição

Preview permite rodar uma aplicação (web, etc.) e Claude vê o resultado.

Objetivos da Aula:

- Entender quando usar Preview
- Rodar aplicação em desenvolvimento
- Usar Preview para testar interativamente

Habilidades Esperadas:

-  Abrir Preview
-  Rodar servidor local
-  Testar aplicação com Claude

Seção 13.2: Quando Usar Preview

Conteúdo:

Preview é útil para:

-  Páginas web (HTML/CSS/JavaScript)
-  Aplicações com interface visual
-  APIs (testar endpoints)
-  Protótipos interativos

NÃO é útil para:

-  Scripts simples (output em terminal)
 -  Processamento de dados (output é número)
 -  Aplicações sem interface visual
-

Seção 13.3: Rodando Preview

Sugestão de Prints:

- Botão "Preview" destacado
- Servidor iniciando (logging)
- Aplicação abrindo em navegador embarcado

Conteúdo:

1. **Peça a Claude:** "Execute em preview"
2. Claude sugere comando ou executa automaticamente
3. **Servidor inicia** (port 3000, 5000, 8000, etc.)
4. **Navegador abre** (dentro Claude Desktop)
5. Você testa aplicação interativamente
6. Claude vê resultado também

Exemplo:



```
Você: "Execute esta app web em preview"

Claude: "Vou executar: npm start"
        [Servidor inicia porta 3000]
        [Preview abre mostrando sua página]

Claude: "Vejo a página. Botão está verde, texto azul.
        Há erro de layout em mobile view."

Você: "Corrija o layout"

Claude: [Modifica CSS]

Você: "Reexecuta preview"

Claude: "Perfeito agora."
```

Pílula Hands-on: Preview uma Aplicação (10 minutos)

Se tem projeto web (HTML/CSS/JS):

1. **Abra projeto em Claude Code**
2. **Peça:** "Execute isto em preview"
3. Claude sugere comando (ex: `python -m http.server 8000`)
4. **Preview abre em lado direito**
5. **Você vê aplicação rodando**
6. **Tire screenshot**

Se não tem projeto web, skip. Volte aqui depois.

MÓDULO 14: SKILLS E COMANDOS

Seção 14.1: Automatizar Tarefas Repetidas

Sugestão de Prints:

- Menu "/" mostrando comandos disponíveis
- Botão "+" para adicionar skill
- Skill sendo executada

Descrição

Skills são tarefas **reutilizáveis**. Define uma vez, usa sempre.

Objetivos da Aula:

- Entender o que é skill
- Usar skills existentes
- Criar skill personalizada

Habilidades Esperadas:

-  Acessar comandos com "/"
-  Usar skill existente
-  Entender poder de automação

Seção 14.2: Usando Skills

Sugestão de Prints:

- Menu "/" aberto mostrando /check, /test, etc
- Skill executada instantaneamente

Conteúdo:

Algumas skills existentes:

- `/check` — verifica se código está pronto
- `/test` — roda testes
- `/lint` — verifica formatação e estilo
- `/doc` — gera documentação

Como usar:

1. Comece a escrever "/" na aba Code
2. Menu mostra skills disponíveis
3. Clique skill desejada
4. Skill é executada

Exemplo:



```
Você digita: /test
Claude: "Vou executar pytest"
      [Testes rodam]
      [Resultado mostrado]
```

Seção 14.3: Criando Skills Pessoais

Conteúdo:

Você pode criar skills customizadas. Exemplo:



```
Skill: /review-code

Descrição: Revisa código e checa:
- Type hints presentes?
- Docstrings presentes?
- Sem erros óbvios?
- Segue padrões do projeto?

Sempre que você digita /review-code, Claude executa esta checklist.
```

Pílula Hands-on: Usar Skills (5 minutos)

1. **Aba Code aberta**
2. **Digite "/" para ver skills disponíveis**
3. **Teste uma skill** (ex: `/test`)
4. **Observe resultado**

MÓDULO 15: MCP E PLUGINS (EXPANDIDO)

Seção 15.1: O Que é MCP?

Sugestão de Prints:

- Logo MCP (Model Context Protocol)
- Diagrama mostrando Claude conectado a sistemas externos
- Exemplos de conexões (GitHub, Jira, BD)

Descrição

MCP (Model Context Protocol) permite Claude se conectar a **sistemas externos**.

Objetivos da Aula:

- Entender o que é MCP
- Conhecer MCPs disponíveis
- Instalar MCP
- Usar MCP para automação

Habilidades Esperadas:

-  Explicar MCP em poucas palavras
-  Conhecer 3-4 MCPs úteis
-  Instalar MCP
-  Usar MCP para tarefas reais

Seção 15.2: MCPs Comuns para Engenharia

Sugestão de Prints:

- Lista de MCPs disponíveis
- Cada MCP com ícone e descrição
- Instalação de MCP destacada

Conteúdo:

MCPs mais úteis:

GitHub MCP

- Claude lê seus repositórios
- Revisa Pull Requests

- Sugere mudanças em issues

Jira MCP

- Claude vê suas tarefas
- Cria issues automaticamente
- Atualiza status de tickets

Database MCP

- Claude consulta seu banco de dados
- Gera relatórios
- Sugere otimizações

File Search MCP

- Claude procura em seus arquivos
- Encontra padrões
- Gera relatórios de conteúdo

Seção 15.3: Instalando MCP

Sugestão de Prints:

- Tela de plugins/MCPs no Claude Desktop
- Botão "+" para adicionar
- MCP sendo instalado
- Confirmação de sucesso

Conteúdo:

1. **Clique em "Plugins"** (ou "MCPs")
2. **Clique "+"** para adicionar novo
3. **Procure pelo MCP desejado** (ex: "GitHub")
4. **Clique Install**
5. **Autentique** (pode abrir navegador para login GitHub, etc.)
6. **MCP está pronto** 

Seção 15.4: Usando MCP (Exemplos Práticos)

Conteúdo:

Exemplo 1: GitHub MCP



Você: "Revise meu pull request no GitHub"

Claude (com GitHub MCP):

"Vou acessar seu repositório..."

PR #42: 'Add user authentication'

- 5 arquivos modificados
- Security:  Parece seguro
- Tests:  Novo código sem testes
- Code review: Sugestões..."

Exemplo 2: Database MCP



Você: "Qual é a query mais lenta no meu banco?"

Claude (com Database MCP):

"Conectando ao seu PostgreSQL..."

Query mais lenta: `SELECT * FROM users JOIN orders...`

Problema: `Missing index` em `users.id`

Sugestão: `CREATE INDEX idx_users_id...`"

Seção 15.5: Diferença entre Plugin e MCP

Conteúdo:

Aspecto	Plugin	MCP
O que é	Extensão do Desktop	Protocolo de conexão
Instala onde	Claude Desktop	Seu computador/servidor
Poder	Moderado	Muito alto
Complexidade	Simples	Técnica
Usa quando	Tarefas simples	Automação avançada

Pílula Hands-on: Explorar MCPs (10 minutos)

1. Abra Claude Desktop
2. Procure seção Plugins/MCPs
3. Veja MCPs disponíveis
4. Leia descrição de 3 MCPs
5. Nota: Qual seria útil para seu projeto?

Se tiver GitHub repo, instale GitHub MCP (avançado).

MÓDULO 16: SESSÕES PARALELAS

Seção 16.1: Múltiplas Linhas de Trabalho

Sugestão de Prints:

- Duas sessões lado a lado
- Projeto em ambas
- Git worktree separados

Descrição

Sessões paralelas permitem trabalhar em **múltiplas tarefas simultaneamente** sem misturar código.

Objetivos da Aula:

- Entender quando usar sessões paralelas
- Criar sessão nova
- Sincronizar sessões

Habilidades Esperadas:

- ☒ Criar sessão nova
- ☒ Trabalhar em múltiplas frentes
- ☒ Entender Git worktrees

Seção 16.2: Quando Usar Sessões Paralelas

Conteúdo:

Use quando:

- ☒ Refatoração AND testes simultaneamente
- ☒ Múltiplos branches em paralelo
- ☒ Documentação enquanto desenvolve
- ☒ Debugging em um, desenvolvimento em outro

Exemplo:



Sessão 1: Refatore main.py
Sessão 2: Crie testes para refatoração
Sessão 3: Atualize documentação

Todas rodando em paralelo. Sem interferência.

Seção 16.3: Criando Sessão Nova

Sugestão de Prints:

- Botão "New Session" ou "+"
- Prompt pedindo nome da sessão
- Nova sessão criada

Conteúdo:

1. **Clique "+" ou "New Session"** (próximo aos nomes de sessão)
2. **Dê nome:** "Session B - Testing"
3. **Selecione mesmo projeto** (ou projeto diferente)
4. **Escolha branch** (se usando Git)
5. Sessão nova está pronta ☒

Seção 16.4: Git Worktrees (Avançado)

Conteúdo:

Se você conhece Git avançado:

Sessões paralelas usam **Git worktrees**. Cada sessão tem seu próprio branch sem interferir.



```
# Sessão 1: Branch feature-1
git worktree list
# /seu-projeto (main)
# /seu-projeto-feature-1 (feature-1)

# Sessão 2: Branch feature-2
# /seu-projeto-feature-2 (feature-2)
```

Não precisa fazer merge até estar pronto.

Pílula Hands-on: Sessões Paralelas (10 minutos)

1. **Abra projeto em Claude Desktop**
2. **Crie Sessão A:** Peça a Claude refatorar [main.py](#)
3. **Crie Sessão B:** Peça a Claude criar testes
4. **Altere entre sessões** (note que não interferem)
5. **Tire screenshot** mostrando duas sessões

MÓDULO 17: PULL REQUESTS E CI (NÍVEL 2)

⚠ **Este é conteúdo AVANÇADO.** Se é iniciante, pule para Módulo 20. Volte aqui após 3-4 semanas usando Claude Desktop.

Seção 17.1: CI/CD com Claude

Descrição

Claude pode monitorar Pull Requests e ajudar com fluxo CI/CD.

Quando Usar:

- Você está em projeto com GitHub Actions
- Quer que Claude revise PRs
- Quer automação de testes/deploy

Seção 17.2: Exemplo Prático

Fluxo:

1. Você faz Push para GitHub
2. GitHub Actions roda testes
3. Testes falham
4. Claude lê erro, sugere correção
5. Você aceita, faz push novamente
6. Testes passam
7. Claude aprova PR, faz merge



MÓDULO 18: TAREFAS AGENDADAS (NÍVEL 2)

 Este é conteúdo **AVANÇADO**. Pule se iniciante.

Seção 18.1: Automação Recorrente

Tarefas agendadas permitem Claude fazer coisas **automaticamente** em horários:

- Toda segunda-feira: revisar código
- Todo dia: rodar testes
- Toda hora: verificar logs

MÓDULO 19: DESKTOP VS CLI

Seção 19.1: Quando Usar Cada Um?

Descrição

Claude existe em duas formas: **Desktop (app)** e **CLI (terminal)**.

Objetivos da Aula:

- Entender diferenças
- Saber quando escolher cada um
- Setup básico de CLI se necessário

Habilidades Esperadas:

- ☒ Explicar Desktop vs CLI
- ☒ Saber quando CLI é melhor
- ☒ Instalar CLI se necessário

Seção 19.2: Comparação

Conteúdo:

Aspecto	Desktop	CLI
Interface	Visual (windows)	Texto (terminal)
Curva aprendizado	Fácil	Média
Integração IDE	Extensão	Plugin
Automação	Manual	Scripts automáticos
Escalabilidade	1 projeto	Múltiplos projetos

Desktop: Para uso interativo, visual, iniciantes

CLI: Para automação, CI/CD, avançado

Seção 19.3: Claude CLI Basics (Se Interessado)

Conteúdo:

CLI funciona em **terminal**. Instalação:



```
# macOS / Linux
pip install anthropic-cli

# Windows
pip install anthropic-cli
```

Uso:



```
claude --code "seu-projeto" "sua tarefa"
```

MÓDULO 20: BOAS PRÁTICAS E RESPONSABILIDADE ACADÊMICA

Seção 20.1: Ética do Uso de IA

Sugestão de Prints:

- Documento de política de integridade acadêmica
- Exemplo de uso correto vs incorreto

Descrição

IA é ferramenta poderosa. Use com responsabilidade.

Objetivos da Aula:

- Entender uso ético de IA
- Declarar uso de IA quando necessário
- Aprender de verdade (não só copiar)

Habilidades Esperadas:

- ☒ Saber quando declarar uso de IA
- ☒ Usar Claude para aprender, não se livrar do aprendizado
- ☒ Aplicar code of conduct acadêmico

Seção 20.2: Usando Claude Responsavelmente

Conteúdo:

☒ Uso Correto:

"Explique o que é recursão"

Claude explica.

Você entende, cria seu próprio código recursivo.

Entrega código seu (entendido completamente).



☒ Uso Incorreto:



"Faça meu trabalho de programação"
Claude gera código.
Você não entende, entrega como é.

Seção 20.3: Declarando Uso de IA

Conteúdo:

Se sua disciplina pede transparência:

Adicione ao relatório/trabalho:



```
---  
## Ferramentas Utilizadas  
  
- Claude Desktop (Anthropic): Usado para explicações, brainstorming,  
  revisão de código. Código foi escrito por mim.  
---
```

Sempre declare:

- ☒ Se foi código gerado por IA (mesmo que modificado)
- ☒ Explicações ajudaram na compreensão
- ☒ IA ajudou em brainstorm

Seção 20.4: Checklist Final

Conteúdo:

Antes de entregar trabalho com ajuda de IA:



```
[ ] Eu entendo cada linha de código?  
[ ] Posso explicar a lógica?  
[ ] Posso refazer código sem IA se pedido?  
[ ] Declarei uso de IA conforme política da disciplina?  
[ ] Código está comentado?  
[ ] Testes cobrem funcionalidade?  
[ ] Nenhuma credencial ou dado sensível exposto?
```

Se respondeu "não" em qualquer, revise antes de entregar.

Pílula Hands-on: Política Pessoal de IA (10 minutos)

Escreva documento:



```
# Minha Política Pessoal de Uso de IA

## Como Vou Usar Claude Desktop

- Para entender conceitos
- Para revisar meu código
- Para sugerir melhorias
- Para debugging

## Como NÃO Vou Usar

- Para entregar código que não entendo
- Para pular aprendizado
- Para não fazer trabalho

## Meu Compromisso

Vou usar IA para aprender melhor, não para evitar aprender.
Vou declarar uso conforme pedido.
Vou ser honesto.

---
Assinado: [seu nome]
Data: [data]
```

ANEXO A: TROUBLESHOOTING & COMMON MISTAKES

A.1: Aplicativo Não Abre

Problema:



```
Instalou, mas clica atalho e nada acontece.
```

Soluções:

1. Reinicie seu computador

- Às vezes é necessário

2. Reinstale (Windows):

Painel de Controle > Programas > Desinstale
Procure "Claude Desktop"
Desinstale
Baixe versão certa novamente (x64 vs ARM64)
Instale

3. Reinstale (macOS):

Finder > Applications
Arraste Claude para Trash
Esvazia Trash
Baixe versão certa (.dmg)
Instale novamente

4. Limpe cache:

- Windows: Delete %APPDATA%\Claude
- macOS: Delete ~/.cache/Claude
- Reinicie

A.2: Erro 403 ou "Upgrade Required"

Problema:

Abra aba Code e aparece "Você precisa fazer upgrade"

Causas & Soluções:

Causa 1: Plano Free

- Solução: Upgrade para Pro ou Max
- Alternativa: Peça ao professor acesso institucional

Causa 2: Acesso não sincronizado

- Solução: Faça logout e login novamente
- Menu > Logout

- Faça login com email Anthropic

Causa 3: Seu plano expirou

- Solução: Renove subscription em account.anthropic.com

A.3: Claude Não Vê Meus Arquivos

Problema:

Abri pasta [em Code](#), mas Claude não vê arquivos



Soluções:

1. Feche e abra novamente:

- Close Code tab
- Abra novamente
- Select project again

2. Selecione pasta certa:

- Você selecionou pasta raiz?
- Ou selecionou arquivo?
- Deve ser pasta (container)

3. Permissões de arquivo:

- No Windows: botão direito pasta > Properties > Security
- Confirme que seu usuário tem "Read" permission

A.4: Terminal Não Funciona

Problema:

Abro terminal e aparece [erro](#) ou não responde



Soluções:

1. Feche e abra novamente:

- Ctrl + (ou [Cmd](#) +)

2. Limpe terminal:

- `clear` (macOS/Linux)
- `cls` (Windows)
- Enter

3. Se Python, tenta:

```
python --version
```

Se retorna versão, Python está ok.

A.5: Git Não Funciona em Windows

Problema:

```
Terminal diz "git não é reconhecido como comando"
```

Solução:

Git não está instalado. Vá para Módulo 2B e instale.

A.6: Claude Quebrou Meu Código

Problema:

```
Claude fez mudança, agora código não funciona
```

Solução Rápida (Git):

```
git diff          # Ver o que mudou  
git checkout .    # Desfazer TODAS mudanças
```

Se é mudança específica:



```
git log           # Ver commits
git revert [hash] # Desfazer um commit
```

A.7: Diff View Mostra Muita Coisa

Problema:



Claude quer mudar 1000 linhas, muito assustador

Solução:

1. **Rejeite mudança grande**
2. **Peça a Claude fazer em passos:**



```
"Não faça tudo. Primeiro, apenas refatore função foo().
Depois, fazemos bar()."
```

3. **Use "Plan mode"** (Módulo 10) para ver plano antes

A.8: Testes Falhando

Problema:



Claude criou código, mas testes quebram

Debug:

1. **Veja erro exacto:**



```
pytest -v
```

2. **Mostre erro a Claude:**



```
"Teste falha com: AssertionError: expected 5, got 4"
```

3. Claude sugere correção

A.9: Token Limite

Problema:



```
Claude diz "Você atingiu token limit"
```

Solução:

- Você atingiu seu limite mensal de tokens
- Espere até mês novo, ou
- Upgrade para plano maior

A.10: Login Não Funciona

Problema:



```
Desktop pede senha, mas não aceita
```

Soluções:

1. Confirme sua senha:

- Tente login em claude.ai
- Se funcionar lá, senha está certa

2. Reset de senha:

- claude.ai > Login > "Esqueceu senha?"
- Reset via email
- Tente novamente no Desktop

3. Autenticação de dois fatores:

- Se ativou 2FA, Desktop pode não apoiar
- Desative 2FA temporariamente (cuidado!)
- Ou use token de API (avançado)

ANEXO B: COMANDOS SEGUROS PARA TERMINAL

Tabela de Referência Rápida

Absolutamente Seguro

Comando	O que faz	Risco
<code>python script.py</code>	Roda script Python	Nenhum (confie no script)
<code>ls</code> / <code>dir</code>	Lista arquivos	Nenhum
<code>pwd</code>	Mostra pasta atual	Nenhum
<code>cat file.txt</code>	Mostra conteúdo	Nenhum
<code>git status</code>	Status do Git	Nenhum
<code>pytest</code>	Roda testes	Nenhum
<code>node script.js</code>	Roda JavaScript	Nenhum
<code>npm test</code>	Roda testes npm	Nenhum

Cuidado (Você Entende Risco)

Comando	O que faz	Risco
<code>pip install lib</code>	Instala pacote	Pode quebrar dependências
<code>npm install</code>	Instala pacotes	Pode quebrar dependências
<code>git commit</code>	Commita mudanças	Grave se commit errado
<code>rm file.txt</code>	Deleta arquivo	Irreversível

Perigoso (Evitar sem Razão)

```
rm -rf folder/      # Deleta pasta inteira
mv file /outro      # Move arquivo
sudo command        # Executa como admin
```



NUNCA Execute



```
sudo rm -rf /      # DELETA TUDO DO COMPUTADOR
eval "code"        # Executa código desconhecido
wget [url]         # Baixa arquivo de Internet
```

ANEXO C: MODELOS DE .claude.md

C.1: Projeto Python Simples



```
# Claude Configuration

## Project Overview
Calculadora de estatísticas de dados. Lê CSV, gera relatórios.

## Coding Standards
- Python 3.10+
- Type hints obrigatórios
- Docstrings em todas as funções
- PEP 8 (use black)

## Testing
- Todos arquivos de teste: test_*.py
- Rodas com: pytest
- Cobertura mínima: 80%

## Tools
- Language: Python 3.10
- Package Manager: pip
- Testing: pytest
- Linting: black, flake8
- VCS: Git

## Important Rules
- Sem hardcode de senhas
- Sem eval() ou exec()
- Sempre error handling
- Logs em arquivo, não console

## Common Commands
```bash
pip install -r requirements.txt
python -m pytest
black .
flake8 .
```



## ## C.2: Projeto Web (JavaScript/Node)

```markdown

Claude Configuration

Project Overview

App web de e-commerce. Frontend React, backend Node.

Frontend (React)

- Componentes funcionais com Hooks
- Styled Components para CSS
- Redux para state management
- Tests com Jest + React Testing Library

Backend (Node/Express)

- Endpoints REST apenas (sem GraphQL)
- Validação com Joi
- Error handling centralizado
- Authentication com JWT

Database

- PostgreSQL
- Migrations com Sequelize
- Seeders para dados de teste

Common Commands

```bash

npm install

npm start # Desenvolvimento

npm test # Testes

npm run build # Produção



## ## C.3: Projeto Misto (Múltiplas Linguagens)

```markdown

Claude Configuration

Project Structure



Backend Python

- FastAPI 0.100+
- Async/await padrão
- Type hints completos
- Pydantic para validação

Frontend React

- Functional components
- Hooks padrão
- Tailwind CSS
- Tests com Vitest

Mobile React Native

- React Native 0.72+
- TypeScript obrigatório
- Expo para build

Diferenças por Pasta

- backend/* : segue Python rules
- frontend/* : segue React rules
- mobile/* : segue React Native rules

Integration Rules

- Backend and Frontend comunicam via REST
- Mobile e Backend, mesma API
- Todas respostas JSON-padronizadas

ENCERRAMENTO DO CURSO

Checklist Final de Aprendizado

Você aprendeu:

- ☐ Instalar e configurar Claude Desktop
- ☐ Entender as três abas (Chat, Code, Cowork)
- ☐ Escolher projeto inicial seguro
- ☐ Usar cada modelo (Haiku, Sonnet, Opus) apropriadamente
- ☐ Escrever prompts efetivos
- ☐ Revisar código em diff view
- ☐ Interromper e redirecionar Claude
- ☐ Fornecer contexto suficiente
- ☐ Entender modos de permissão
- ☐ Usar terminal integrado seguramente
- ☐ Proteger dados sensíveis (.env, .gitignore)

- ☐ Integrar com sua IDE
- ☐ Configurar .claude.md
- ☐ Usar Git com Claude
- ☐ Conhecer MCP e plugins
- ☐ Trabalhar com sessões paralelas
- ☐ Usar Claude responsavelmente

Próximos Passos

1. **Semana 1:** Conforto com Módulos 0-8 (básico)
2. **Semana 2-3:** Domínio de Módulos 9-14 (intermediário)
3. **Semana 4+:** Explorar Módulos 15-16 (avançado)
4. **Depois:** Módulos 17-18 (nível 2), MCP (automação)

Mentora Final

Claude Desktop não é máquina mágica. É **ferramenta de amplificação**.

Amplifica seu entendimento se você entende.

Amplifica sua preguiça se não estudar.

Escolha amplificar aprendizado, não preguiça.

Sucesso em sua jornada! 🚀

Fim do Roteiro

Versão 2.0 — Reformulado com base em auditoria crítica de pedagogia e execução técnica. Todos os problemas identificados foram corrigidos.

Para feedback: contact@anthropic.com